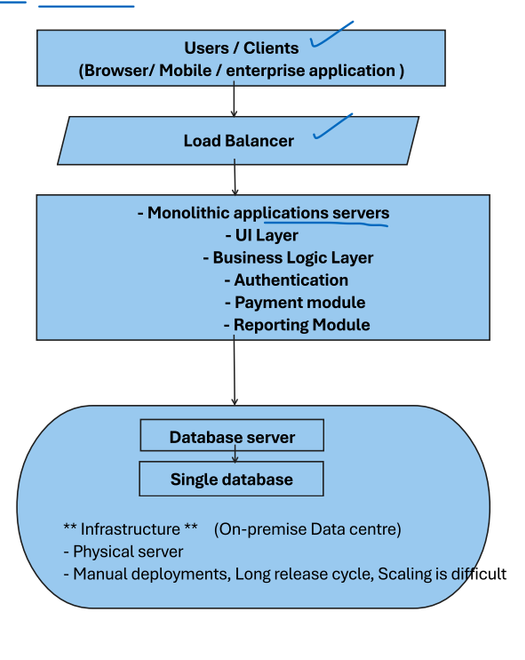
*** GCP_Containerization ***

Introduction to Containers & Google Cloud

1. Overview of Modernization Trends

Concept: Organizations are moving from monolithic, on-premise applications to cloud-native architectures.

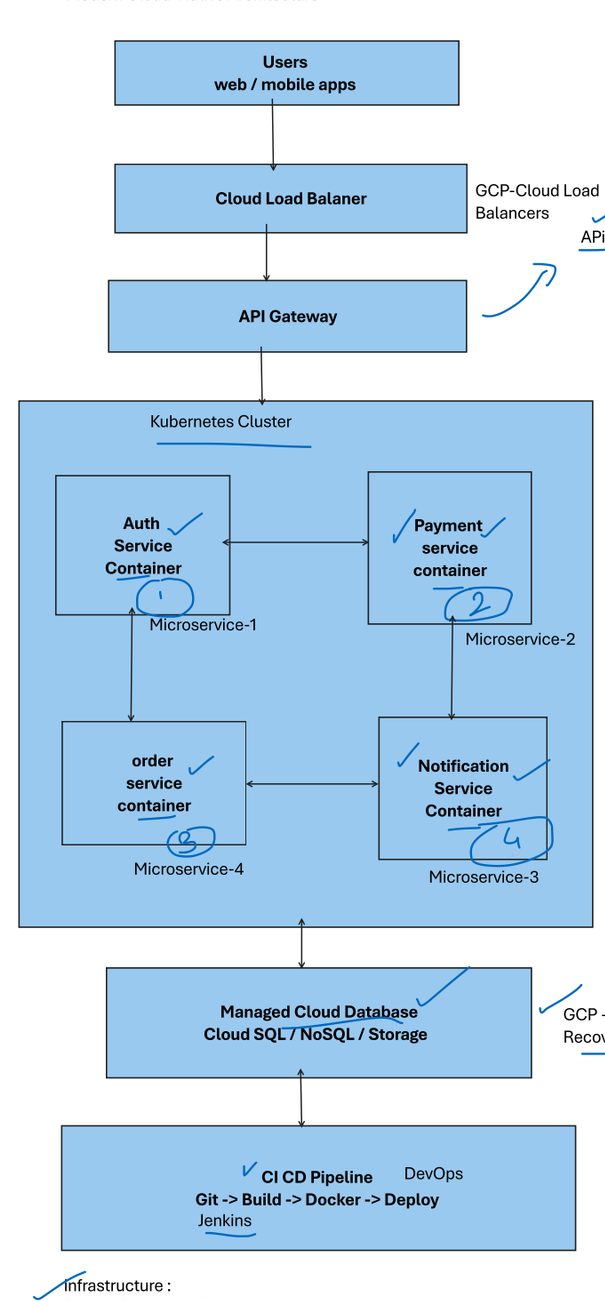
Traditional Architecture:
- Physical servers
- Monolithic applications
- Manual deployments
- Long release cycles


Users / Clients (Browser/ Mobile / enterprise application)
Load Balancer
- Monolithic applications servers
- UI Layer
- Business Logic Layer
- Authentication
- Payment module
- Reporting Module
Database server
Single database

** Infrastructure ** (On-premise Data centre)
- Physical server
- Manual deployments, Long release cycle, Scaling is difficult

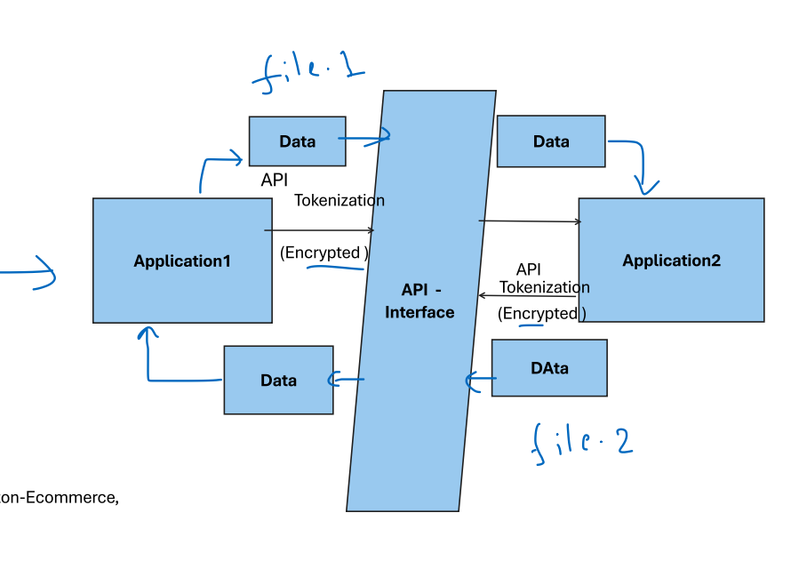
Key Problems
- Entire application must be redeployed for small changes
- Scaling requires bigger servers
- Slow development cycles
- High infrastructure cost

** Modern Cloud-Native Architecture **

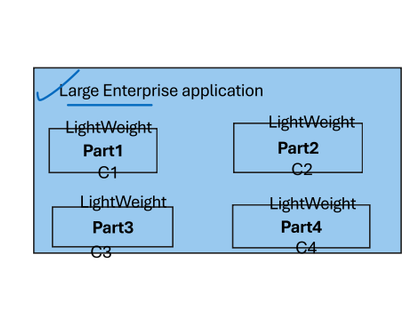

Users
web / mobile apps
Cloud Load Balaner
API Gateway
Kubernetes Cluster
Auth Service Container
1
Microservice-1
Payment service container
2
Microservice-2
order service container
3
Microservice-4
Notification Service Container
4
Microservice-3
Managed Cloud Database
Cloud SQL / NoSQL / Storage
CI CD Pipeline
DevOps
Git -> Build -> Docker -> Deploy
Jenkins

GCP-Cloud Load Balancers

APi - SAFE & High Security


file-1
Data
API
Tokenization
(Encrypted)
Application1
Data
API - Interface
Data
Application2
API
Tokenization
(Encrypted)
DAta
file-2

E.g: Container apps -Amazon-Ecommerce, REDBUS service etc


Large Enterprise application
LightWeight
Part1
C1
LightWeight
Part2
C2
LightWeight
Part3
C3
LightWeight
Part4
C4

GCP - databases - More Felxibility - Auto - Scaling, Failure Recovery mechanism , low cost (Serverless)

VCS - Version Control system Tools: Git / Github / Gitlab / BitBucket

Jenkins , Google Cloud - Build

Infrastructure :
Google Cloud / AWS/ Azure/
Containers + Kubernetes
Auto scaling
High Availability

Key Transformation:

Traditional	Modern
Monolithic App	Microservices
Physical Servers	Cloud Infrastructure
Manual Deployments	CI/CD Automation
VM-based apps	Containers
Vertical Scaling	Horizontal Scaling

Simple CI/CD Pipeline Architecture

Developer
↓
Source Code Repository
(GitHub / GitLab)
↓
CI Tool
(Jenkins / GitHub Actions / GitLab CI)
↓
Build + Test
↓
Container Image
(Docker)
↓
Container Registry
↓
CD Tool
(ArgoCD / Spinnaker)
↓
Kubernetes Deployment

**Key Drivers: **

- Scalability
- Faster deployments
- High availability
- DevOps automation

** Real-World Example **

Netflix migrated from monolithic architecture to microservices running in containers to handle millions of users.

1. Traditional Architecture at Netflix (Earlier)

Initially, Netflix ran a large monolithic application in a traditional data center.

** Characteristics **

Single large Java application

Hosted on physical servers

Manual scaling

- Single point of failure

Difficult deployments

** Problems Faced **

❌ Hard to scale during high traffic
❌ Any small update required redeploying the entire system
❌ Outages affected the entire platform
❌ Infrastructure limits in data centers

Example

When a new movie release caused traffic spikes, the system struggled to handle millions of users.

** Architecture (Earlier) **
Users
↓
Load Balancer
↓
Monolithic Application Server
↓
Database

2. Modern Cloud-Native Architecture **

Netflix migrated to microservices running in containers on cloud infrastructure.

Key Technologies Used:
- Microservices architecture
- Containers
- Cloud infrastructure
- DevOps automation
- Distributed systems

*** Architecture (Modern) ***

Users
↓
Global CDN (Google Cloud CDN Service)- Content Delivery Network - High-Performance Google Network
↓
API Gateway
↓
Microservices Layer
├── User Service
├── Recommendation Service
├── Billing Service
├── Streaming Service
└── Content Metadata Service
↓
Databases + Caches

Each service runs independently in containers.

** 3. Advantages Netflix Achieved:

1. Massive Scalability **

Netflix can handle hundreds of millions of users globally.

Example
During major releases or peak hours, services automatically scale.

2. Faster Deployments **

Before modernization:

Few releases per month

After modernization:

Thousands of deployments per day

3. Fault Isolation **

If Recommendation Service fails, the rest of the platform continues running.

4. Global Availability **

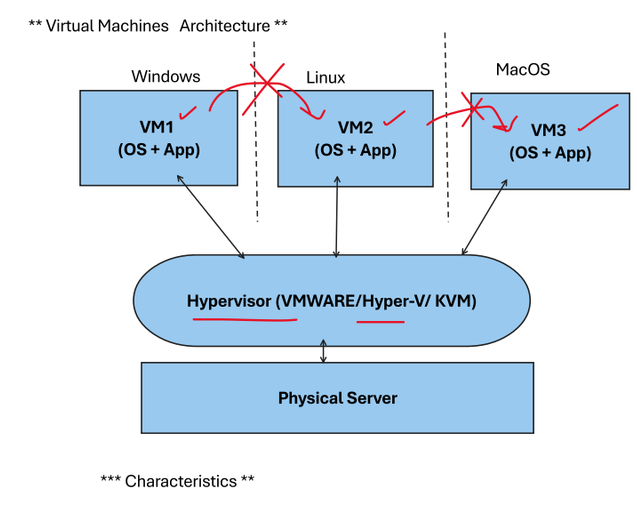
Services are deployed across multiple regions worldwide.

Users are automatically routed to the nearest region.

*** Containers vs Virtual Machines **

Feature	Containers	Virtual Machines
OS	Share the host operating system	Each VM has its own full OS
Size	Lightweight (MBs)	Heavy (GBs)
Boot Time	Starts in seconds	Takes minutes
Resource Usage	Low resource consumption	High resource usage
Isolation	Process-level isolation	Hardware-level isolation

** Virtual Machines Architecture **


Windows
VM1 (OS + App)
Linux
VM2 (OS + App)
MacOS
VM3 (OS + App)
Hypervisor (VMWARE/Hyper-V/ KVM)
Physical Server

*** Characteristics **

Each VM includes:
- Guest OS
- Application
- Libraries

Higher resource consumption

Strong isolation

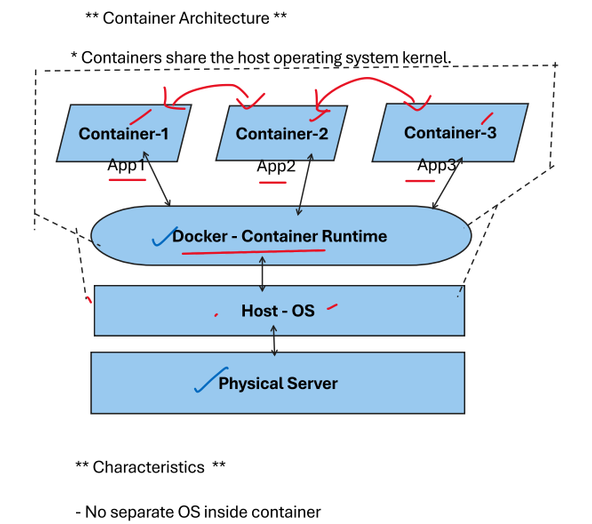
Example: Running 5 applications requires: 5 Operating Systems - 5 VM instances

VM Deployment

App1 → VM → OS → Hardware
App2 → VM → OS → Hardware
App3 → VM → OS → Hardware

** Container Architecture **

* Containers share the host operating system kernel.


Container-1
App1
Container-2
App2
Container-3
App3
Docker - Container Runtime
Host - OS
Physical Server

** Characteristics **

- No separate OS inside container

- Very lightweight

- Faster startup time

- High density (many containers per host)

Container Deployment

App1 → Container → Host OS
App2 → Container → Host OS
App3 → Container → Host OS

** Performance Comparison **

Aspect	Containers	Virtual Machines
Startup Time	Seconds	Minutes
Memory Usage	Low	High
Deployment Speed	Very fast	Slow
Scalability	High	Moderate

When to Use Virtual Machines:

Virtual machines are better for:

Running different operating systems

Strong security isolation

Legacy applications

Compliance-heavy environments

When to Use Containers

Containers are ideal for:

Microservices architecture

DevOps CI/CD pipelines

Cloud-native applications

Kubernetes deployments

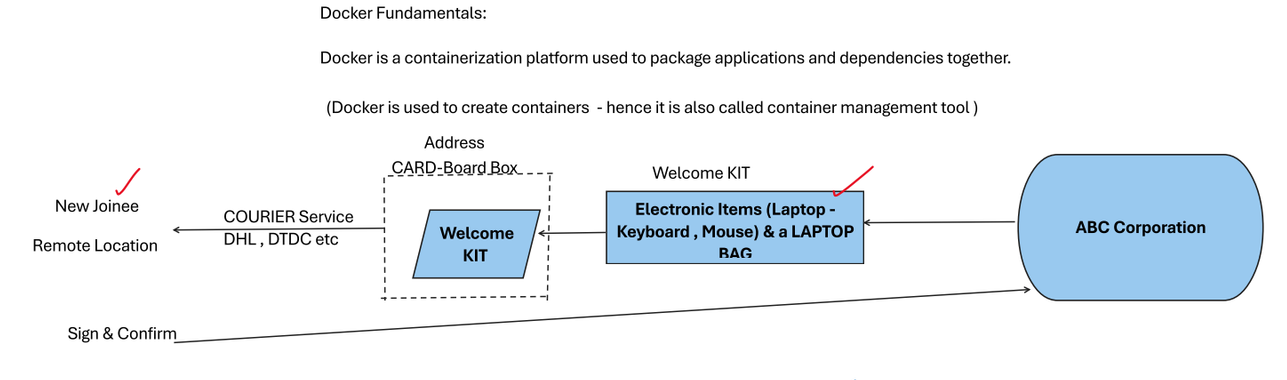
Running 10 applications:

VM approach → 10 OS
Container approach → 1 OS

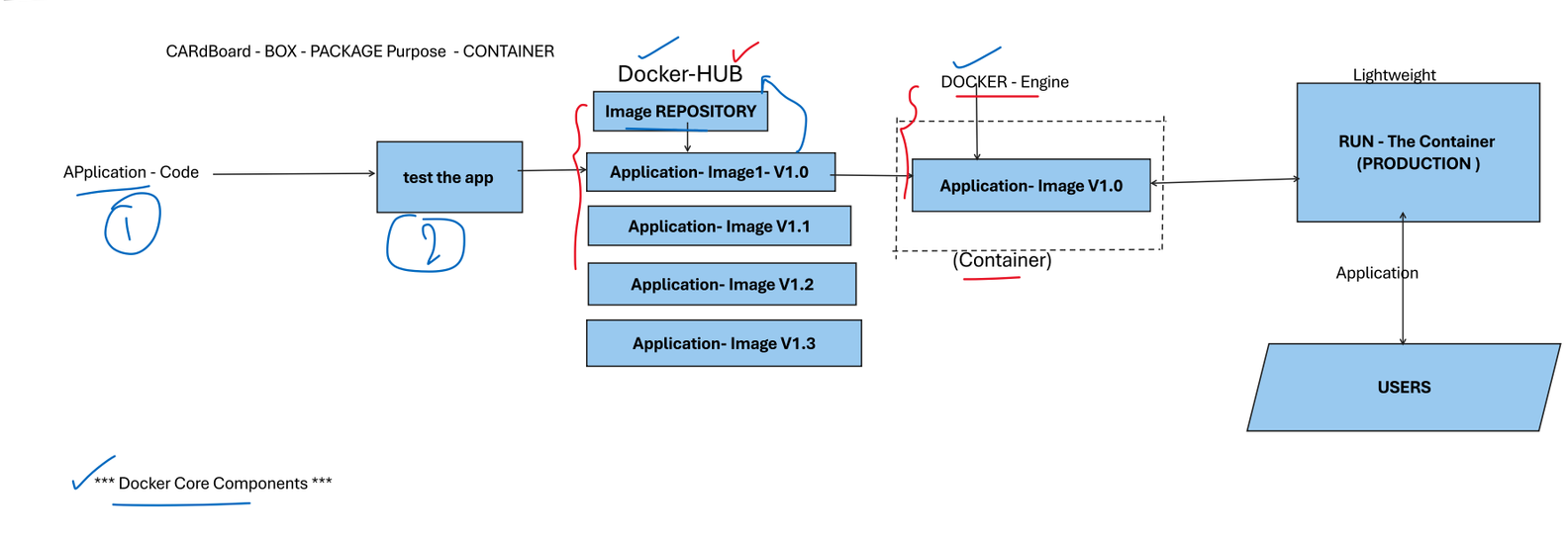
** Docker **
Docker Fundamentals:

Docker is a containerization platform used to package applications and dependencies together.

(Docker is used to create containers - hence it is also called container management tool)

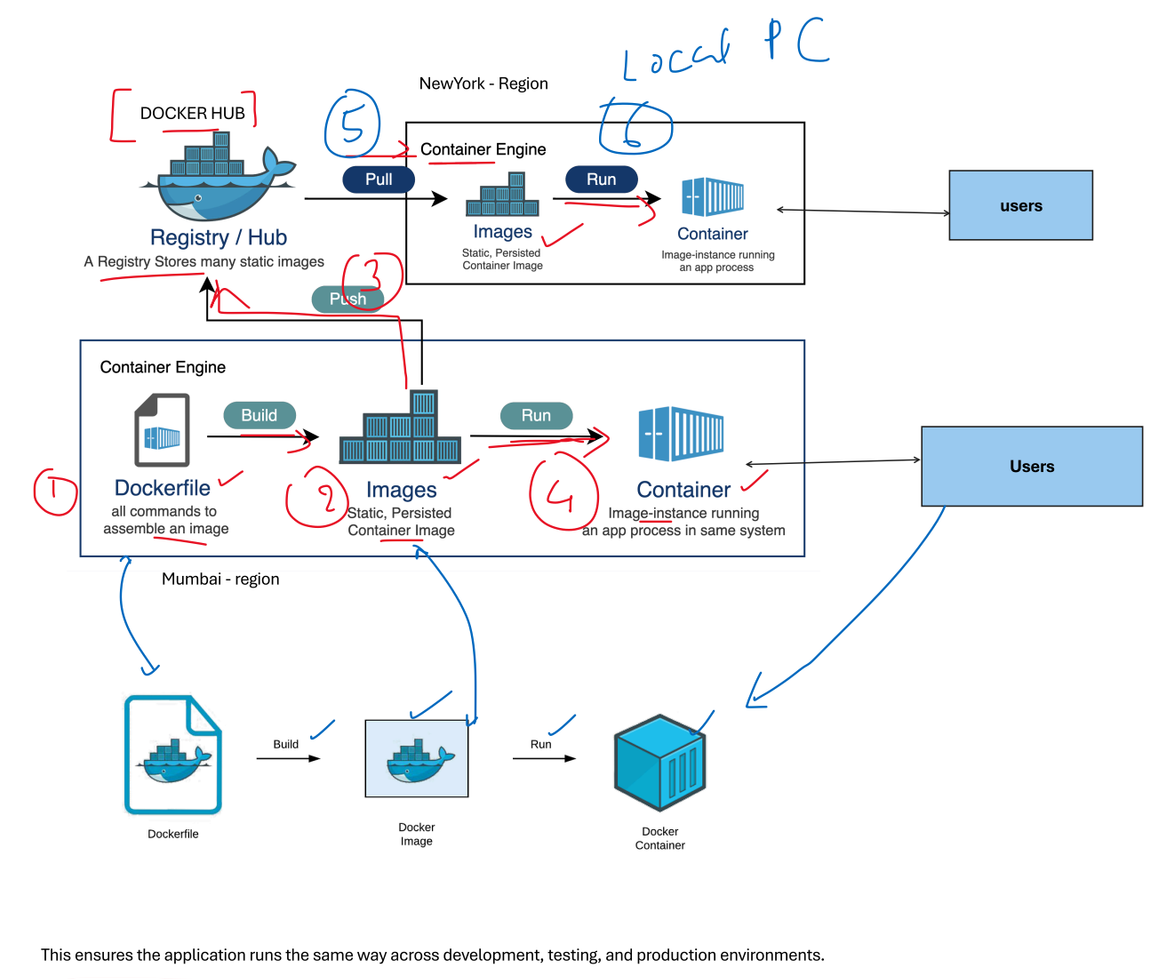

New Joinee
Remote Location
COURIER Service
DHL , DTDC etc
Address
CARD-Board Box
Welcome KIT
Welcome KIT
Electronic Items (Laptop - Keyboard , Mouse) & a LAPTOP BAG
ABC Corporation
Sign & Confirm

CARdBoard - BOX - PACKAGE Purpose - CONTAINER


APplication - Code
1
test the app
2
Docker-HUB
Image REPOSITORY
Application- Image1- V1.0
Application- Image V1.1
Application- Image V1.2
Application- Image V1.3
DOCKER - Engine
Application- Image V1.0
(Container)
Lightweight
RUN - The Container (PRODUCTION)
Application
USERS

*** Docker Core Components ***

Component	Description
Docker Engine	Runtime that runs containers
Docker Image	Blueprint of container
Docker Container	Running instance of image
Dockerfile	Script to build image
Docker Hub	Image repository


DOCKER HUB
Registry / Hub
A Registry Stores many static images
5
Pull
Push
3
NewYork - Region
Local PC
6
Container Engine
Images
Static, Persisted Container Image
Run
Container
Image-instance running an app process
users
Container Engine
Build
Run
1
Dockerfile
all commands to assemble an image
2
Images
Static, Persisted Container Image
4
Container
Image-instance running an app process in same system
Users
Mumbai - region
Dockerfile
Build
Docker Image
Run
Docker Container

This ensures the application runs the same way across development, testing, and production environments.

Example problem Docker solves:

Developer Machine → Works

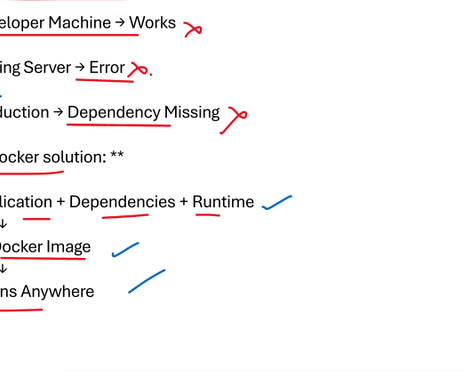
Testing Server → Error

Production → Dependency Missing

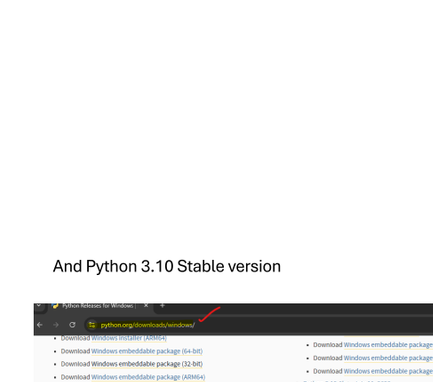
** Docker solution: **

Application + Dependencies + Runtime
↓
Docker Image
↓
Runs Anywhere

VSCode :


Download Visual Studio Code

And Python 3.10 Stable version



- Please create a case to - technical team - Mention that :

- You need to install VScode : version 1.111

- Purpose: GCP - Docker - Containerization Training & coding

** Docker Architecture (Simple Block Diagram) **

Developer
↓
Docker CLI
↓
Docker Engine
↓
Container 1 (App) | Container 2 (App) | Container 3 (App)

** Docker Workflow **

Application Code
↓
Dockerfile
↓
Docker Build
↓
Docker Image
↓
Docker Container

Explanation:

1. Write application code

2. Create a Dockerfile

3. Build a Docker image

4. Run the container

users

** Create Dockerfile **
Example Dockerfile:

```
FROM python:3.9

WORKDIR /app

COPY . .

RUN pip install flask

CMD ["python","app.py"]
```

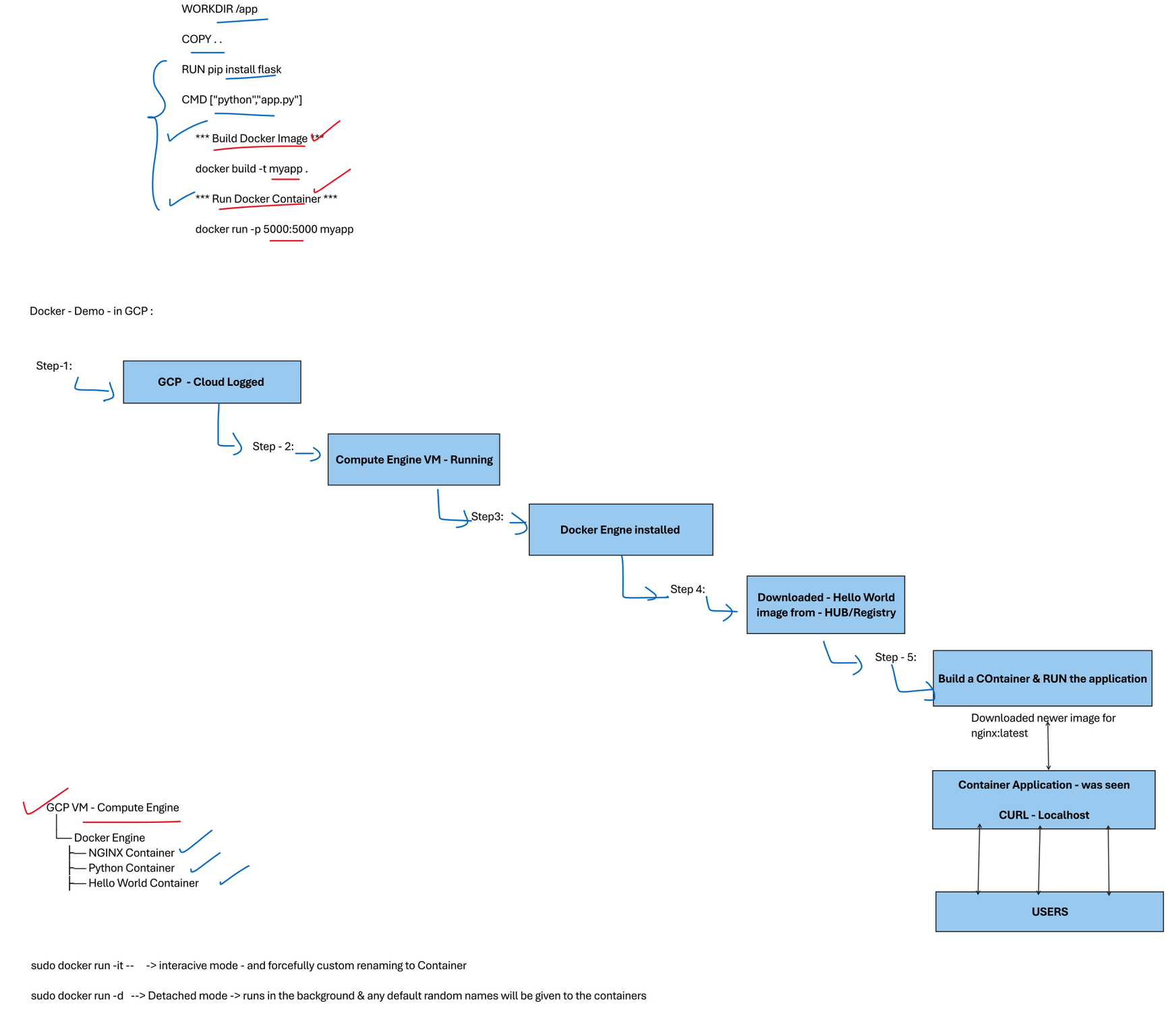
*** Build Docker Image ***

```
docker build -t myapp .
```

*** Run Docker Container ***

```
docker run -p 5000:5000 myapp
```

Docker - Demo - in GCP :


Step-1:
GCP - Cloud Logged
Step - 2:
Compute Engine VM - Running
Step3:
Docker Engne installed
Step 4:
Downloaded - Hello World image from - HUB/Registry
Step - 5:
Build a COntainer & RUN the application
Downloaded newer image for nginx:latest
Container Application - was seen
CURL - Localhost
USERS

GCP VM - Compute Engine
└─ Docker Engine
  ├─ NGINX Container
  ├─ Python Container
  └─ Hello World Container

sudo docker run -it -- -> interacive mode - and forcefully custom renaming to Container

sudo docker run -d --> Detached mode -> runs in the background & any default random names will be given to the containers

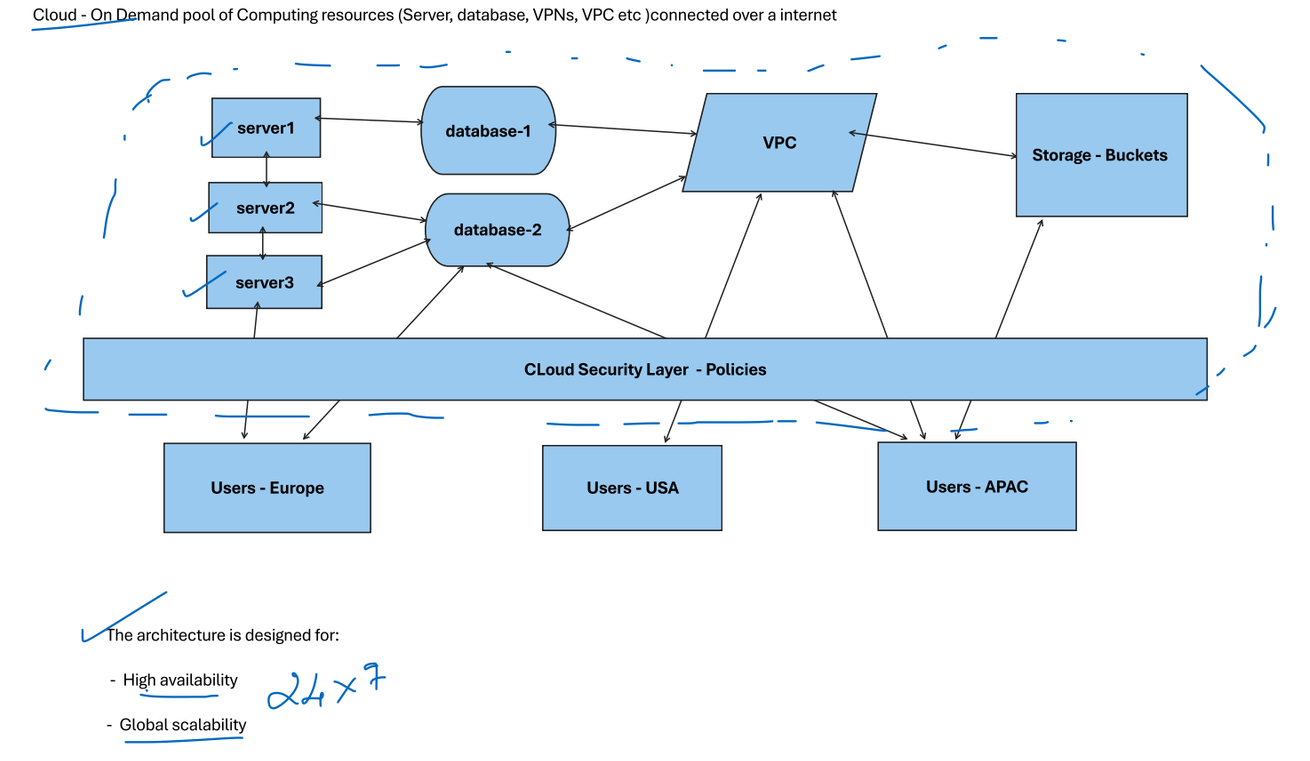
** Very Common Docker Comands: **

```
docker pull nginx
docker images
docker run -d -p 80:80 --name web1 nginx
docker ps
docker ps -a
docker logs web1
docker exec -it web1 bash
docker stop web1
docker rm web1
docker inspect nginx
```

*** Introduction to Google Cloud Architecture ***

Google Cloud Platform (GCP) provides a global Cloud infrastructure that allows organizations to run applications, store data, and build scalable cloud solutions.

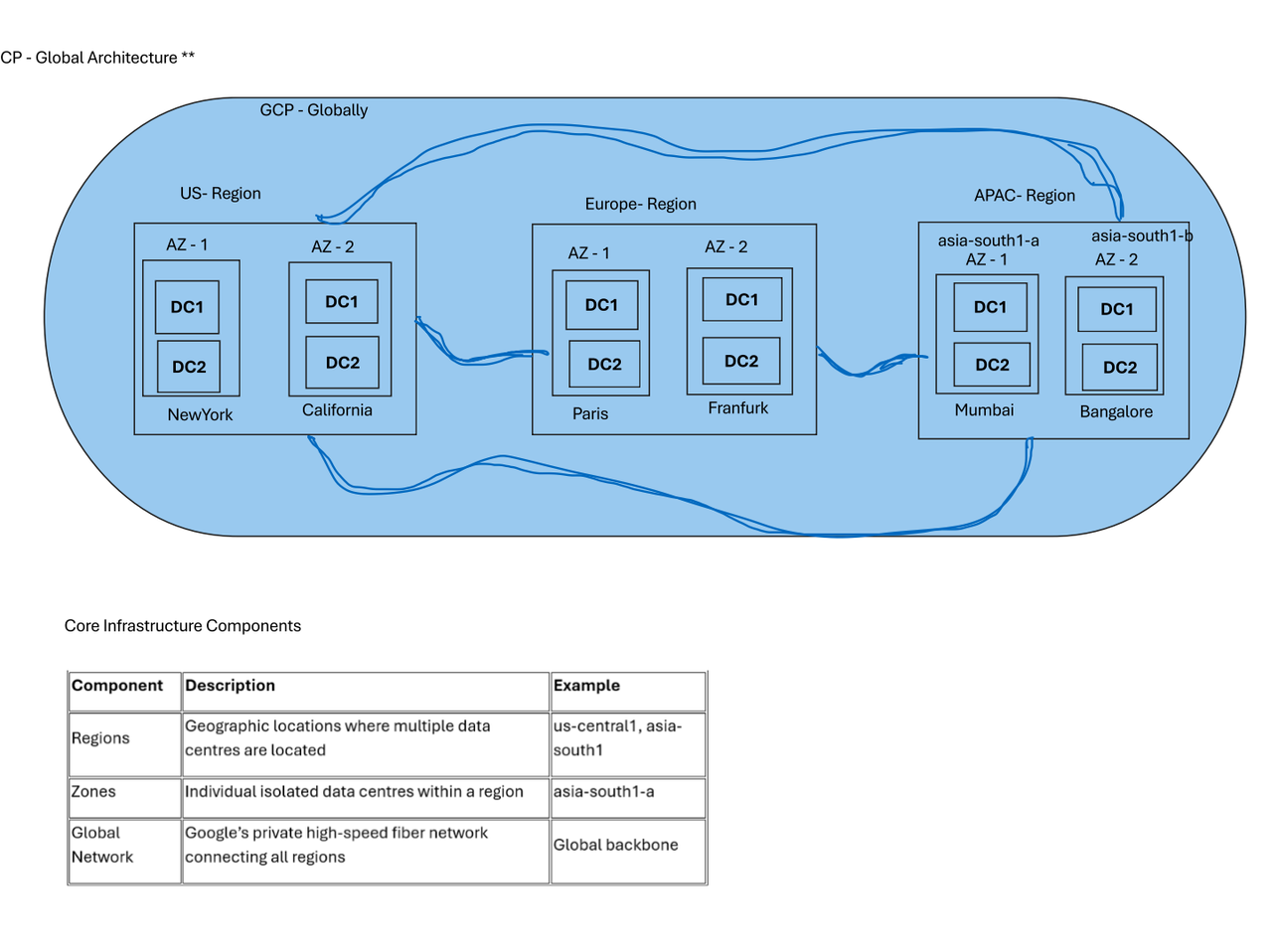
Cloud - On Demand pool of Computing resources (Server, database, VPNs, VPC etc)connected over a internet


server1
server2
server3
database-1
database-2
VPC
Storage - Buckets
CLoud Security Layer - Policies
Users - Europe
Users - USA
Users - APAC

The architecture is designed for:

- High availability 24x7
- Global scalability
- Low latency networking

** GCP - Global Architecture **


GCP - Globally
US- Region
AZ - 1
AZ - 2
DC1
DC2
NewYork
California
Europe- Region
AZ - 1
AZ - 2
DC1
DC2
Paris
Franfurk
APAC- Region
asia-south1-a
AZ - 1
asia-south1-b
AZ - 2
DC1
DC2
Mumbai
Bangalore

Core Infrastructure Components

Component	Description	Example
Regions	Geographic locations where multiple data centres are located	us-central1, asia-south1
Zones	Individual isolated data centres within a region	asia-south1-a
Global Network	Google's private high-speed fiber network connecting all regions	Global backbone

1. Regions:

A Region is a specific geographic location where Google operates cloud infrastructure.
Each region contains multiple zones to ensure redundancy and fault tolerance.

Example regions:

* us-central1 (Iowa, USA)

* asia-south1 (Mumbai, India)

* europe-west1 (Belgium)

Why Regions Are Important

Regions help organizations achieve:

- Low latency for users located nearby

- Data residency compliance for legal requirements

- Disaster recovery strategies

- High availability across geographic locations

* Example: A company serving customers in India would deploy workloads in asia-south1 (Mumbai) to reduce network latency.

2. Zones

A Zone is an isolated data center inside a region.
Each region typically contains three or more zones.

Example:

Region	Zones
asia-south1	asia-south1-a
	asia-south1-b
	asia-south1-c

Benefits of Zones:
Fault isolation – if one zone fails, others continue operating

High availability – applications can run across multiple zones

Resilient architecture – improves reliability of services

If Zone A fails, traffic can automatically move to Zone B, ensuring uninterrupted service.

3. Global Network:

Google operates one of the world's largest private fiber networks, which connects all cloud regions.
- This global backbone network provides:
- Low-latency communication between regions
- Secure data transmission
- High-speed connectivity across continents

This same infrastructure supports large-scale services like:
-YouTube
-Gmail
-Google Search

* GCP Resource Hierarchy **
Resources in Google Cloud follow a hierarchical structure.

Organization — Top-level company account
↓
Folders — Logical grouping of projects
↓
Projects — Main working unit where resources exist
↓
Resources — Services like VMs, storage, databases

Example:

Company-ABC Corp
↓
IT Department
↓
Project: ecommerce-app
↓
Resources:
├── Compute Engine VM
├── Cloud Storage Bucket
└── Cloud SQL

** IAM (Identity and Access Management) **
- Google Cloud IAM controls who can access which resources in Google Cloud.
- IAM follows the principle of least privilege.

** IAM Components **

Component	Description
Principal	User, group, or service account
Role	Set of permissions
Resource	Google Cloud service

Example services: Google Compute Engine, Google Cloud Storage

IAM Example:

Component	Example
Principal	DevOps Engineer
Role	Compute Admin
Resource	VM Instances

Example meaning:

DevOps Engineer
↓
Assigned Role: Compute Admin
↓
Can manage VM instances in the project

** project setup in GCP **

A GCP Project is a logical container that holds:

- Compute resources

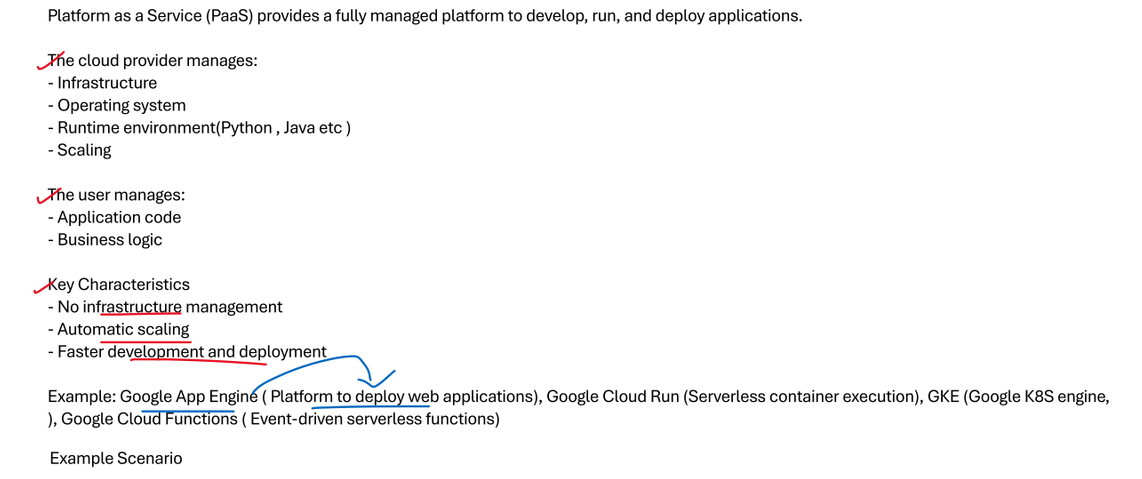
- Storage services

- Databases

- Networking configurations ,APIs and services

It also acts as the unit for billing, permissions, and resource management.

Cloud services in Google Cloud Platform are generally categorized into IaaS, PaaS, and SaaS, depending on how much infrastructure management the user is responsible for.


IaaS
PaaS
SaaS

1. IaaS (Infrastructure as a Service):

Infrastructure as a Service (IaaS) provides virtualized computing resources over the internet.

The cloud provider manages:
- Physical servers
- Networking
- Data centers
- Storage hardware

The user manages:
- Operating system
- Applications
- Runtime environment
- Security configurations

** Key Characteristics**
- Full control over virtual machines
- Flexible infrastructure management
- Suitable for custom applications

Example: Google Compute Engine , Google Cloud Storage , Google Virtual Private Cloud etc

2. PaaS (Platform as a Service)

Platform as a Service (PaaS) provides a fully managed platform to develop, run, and deploy applications.

The cloud provider manages:
- Infrastructure
- Operating system
- Runtime environment(Python , Java etc)
- Scaling

The user manages:
- Application code
- Business logic

Key Characteristics
- No infrastructure management
- Automatic scaling
- Faster development and deployment

Example: Google App Engine (Platform to deploy web applications), Google Cloud Run (Serverless container execution), GKE (Google K8S engine,), Google Cloud Functions (Event-driven serverless functions)

Example Scenario

A developer deploys a Python web application to App Engine (It is for Algo-trading), and Google automatically handles:
- Scaling
- Infrastructure
- Load balancing

- In that app - if Conditions are met then it will trigger events

3. SaaS (Software as a Service)

Software as a Service (SaaS) delivers fully managed software applications over the internet.

The cloud provider manages everything:
- Infrastructure
- Platform
- Application
- Security updates

Users simply access the application through a browser.

Key Characteristics:
- No infrastructure management
- Subscription-based usage
- Accessible from anywhere

Examples from Google Ecosystem:

Service	Description
Google Docs	Online document editor
Gmail	Email service
Google Drive	Cloud file storage

Comparison of IaaS, PaaS, SaaS:

Feature	IaaS	PaaS	SaaS
Infrastructure Management	User	Cloud Provider	Cloud Provider
OS Management	User	Cloud Provider	Cloud Provider
Application Control	Full	Partial	None
Flexibility	High	Medium	Low
Example	Compute Engine	App Engine	Gmail

Simple Real-World Analogy

Cloud Model	Example
IaaS	Renting a house and managing everything inside
PaaS	Renting a furnished apartment
SaaS	Staying in a hotel